

```

#include <iostream>
#include <queue>
#include <vector>
#include <string>
using namespace std;
struct Patient {
    int id;
    string name;
    string arrivalTime;
    int severity;
    string doctor;
    int arrivalOrder;
};
// Higher severity gets higher priority.
// If severity is same, earlier arrival gets higher priority.
struct Compare {
    bool operator()(const Patient& a, const Patient& b) {
        if (a.severity != b.severity)
            return a.severity < b.severity;
        return a.arrivalOrder > b.arrivalOrder;
    }
};
priority_queue<Patient, vector<Patient>, Compare> pq;
int arrivalCounter = 0;
// Add patient
void addPatient() {
    Patient p;

```

```

cout << "\nEnter Patient ID: ";
cin >> p.id;
cout << "Enter Patient Name: ";
cin >> p.name;
cout << "Enter Arrival Time: ";
cin >> p.arrivalTime;
cout << "Enter Severity (1-5): ";
cin >> p.severity;
if (p.severity < 1 || p.severity > 5) {
    cout << "Invalid severity level!\n";
    return;
}
cout << "Enter Assigned Doctor: ";
cin >> p.doctor;
p.arrivalOrder = ++arrivalCounter;

pq.push(p);
cout << "Patient added successfully!\n";
}

// View next patient
void viewNextPatient() {
    if (pq.empty()) {
        cout << "\nNo patients waiting.\n";
        return;
    }
    Patient p = pq.top();
    cout << "\nNext Patient for Treatment\n";

```

```

    cout << "Patient ID: " << p.id << endl;
    cout << "Name: " << p.name << endl;
    cout << "Arrival Time: " << p.arrivalTime << endl;
    cout << "Severity: " << p.severity << endl;
    cout << "Doctor: " << p.doctor << endl;
}

// Treat patient
void treatPatient() {
    if (pq.empty()) {
        cout << "\nNo patients waiting.\n";
        return;
    }
    Patient p = pq.top();
    pq.pop();
    cout << "\nPatient Treated Successfully\n";
    cout << "Patient ID: " << p.id << endl;
    cout << "Name: " << p.name << endl;
    cout << "Severity: " << p.severity << endl;
}

// Search patient
void searchPatient() {
    if (pq.empty()) {
        cout << "\nNo patients waiting.\n";
        return;
    }
    int id;
    cout << "\nEnter Patient ID to search: ";

```

```

cin >> id;

priority_queue<Patient, vector<Patient>, Compare> temp = pq;

bool found = false;

while (!temp.empty()) {
    Patient p = temp.top();
    temp.pop();
    if (p.id == id) {
        cout << "\nPatient Found\n";
        cout << "Patient ID: " << p.id << endl;
        cout << "Name: " << p.name << endl;
        cout << "Arrival Time: " << p.arrivalTime << endl;
        cout << "Severity: " << p.severity << endl;
        cout << "Doctor: " << p.doctor << endl;
        found = true;
        break;
    }
}

if (!found)
    cout << "Patient not found.\n";
}

// Display all waiting patients
void displayPatients() {
    if (pq.empty()) {
        cout << "\nNo patients waiting.\n";
        return;
    }
    priority_queue<Patient, vector<Patient>, Compare> temp = pq;

```

```

cout << "\nPatients Waiting for Treatment\n";
while (!temp.empty()) {
    Patient p = temp.top();
    temp.pop();
    cout << "\nPatient ID: " << p.id;
    cout << "\nName: " << p.name;
    cout << "\nSeverity: " << p.severity;
    cout << "\nArrival Order: " << p.arrivalOrder;
    cout << "\nDoctor: " << p.doctor << endl;
}
}

// Update doctor information
void updateDoctor() {
    if (pq.empty()) {
        cout << "\nNo patients waiting.\n";
        return;
    }
    int id;
    string newDoctor;
    cout << "\nEnter Patient ID: ";
    cin >> id;

    cout << "Enter New Doctor: ";
    cin >> newDoctor;
    priority_queue<Patient, vector<Patient>, Compare> temp;
    bool found = false;

```

```

while (!pq.empty()) {
    Patient p = pq.top();
    pq.pop();
    if (p.id == id) {
        p.doctor = newDoctor;
        found = true;
    }
    temp.push(p);
}
pq = temp;
if (found)
    cout << "Doctor information updated successfully.\n";
else
    cout << "Patient not found.\n";
}

int main() {
    int choice;
    do {
        cout << "\n=====";
        cout << "\n EMERGENCY PATIENT MANAGEMENT SYSTEM";
        cout << "\n=====";
        cout << "\n1. Register Patient";
        cout << "\n2. View Next Patient";
        cout << "\n3. Treat Patient";
        cout << "\n4. Search Patient";
        cout << "\n5. Update Doctor";
    } while (choice != 0);
}

```

```
cout << "\n6. Display All Patients";  
cout << "\n7. Exit";  
cout << "\nEnter your choice: ";  
cin >> choice;  
switch (choice) {  
    case 1:  
        addPatient();  
        break;  
    case 2:  
        viewNextPatient();  
        break;  
    case 3:  
        treatPatient();  
        break;  
    case 4:  
        searchPatient();  
        break;  
    case 5:  
        updateDoctor();  
        break;  
    case 6:  
        displayPatients();  
        break;  
    case 7:  
        cout << "\nThank you!\n";  
        break;  
    default:
```

```
        cout << "\nInvalid choice!\n";  
    }  
} while (choice != 7);  
return 0;  
}
```